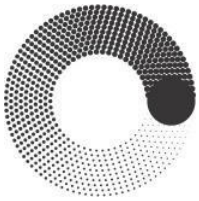
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ



МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет Информационных технологий

Кафедра Информатики и информационных технологий

Направление подготовки

09.03.01 «Информатика и вычислительная техника»

Лабораторная работа №3

Дисциплина: Автоматизация процессов жизненного цикла программных средств

Тема: «Контейнеризация»

Разработчики, ст. гр. 241–327:

- Ночной Максим Сергеевич
- Пахалюк Илья Николаевич
- Сапрыкин Пётр Иванович

Москва 2026

Оглавление

ПЛАН ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ 3

Часть 1.....3

Часть 2.....7

ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ 12

ВОПРОСЫ ДЛЯ ИЗУЧЕНИЯ.....13

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....14

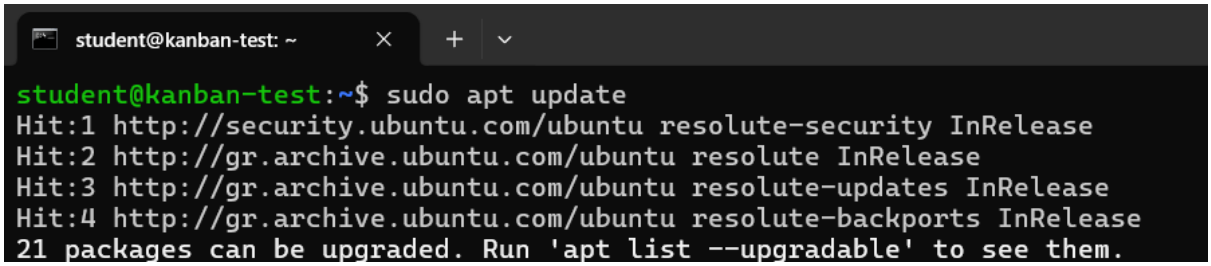
ПЛАН ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Часть 1

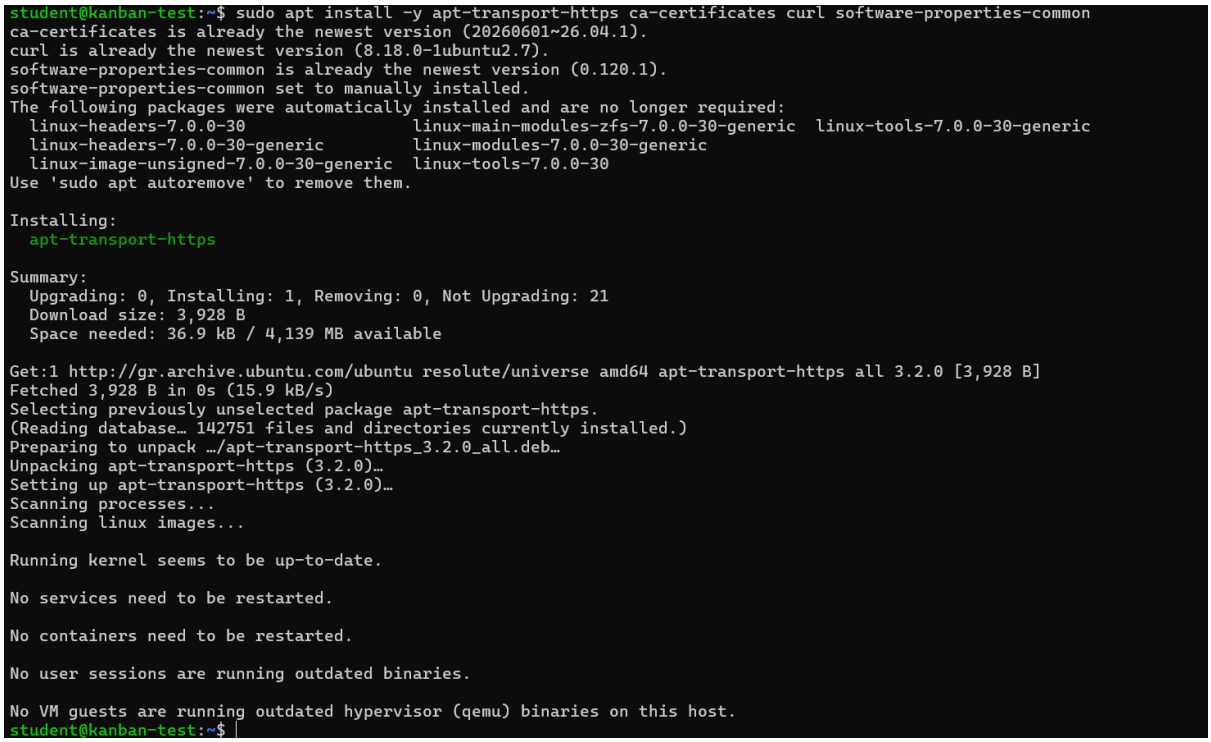
1. Установить Docker на всех развернутых в ходе ЛР2 виртуальных машинах.

Для стендов проекта «KanbanBoard» используем Docker Engine. Установка выполняется через SSH на виртуальных машинах kanban-test, kanban-stage и kanban-prod.

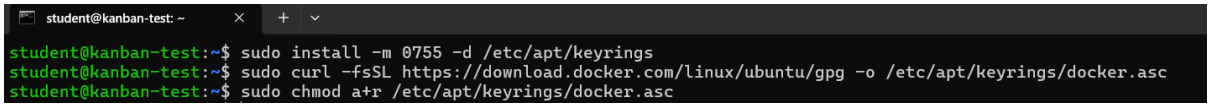
Обновим существующий список пакетов:



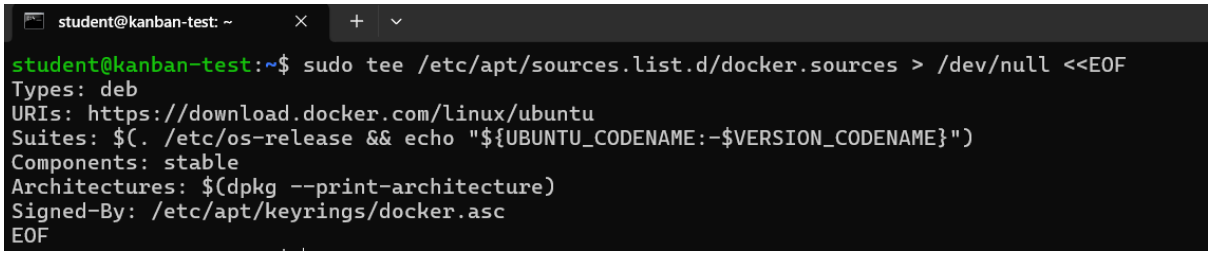
Затем установим необходимые пакеты, которые позволяют APT использовать репозитории через HTTPS:



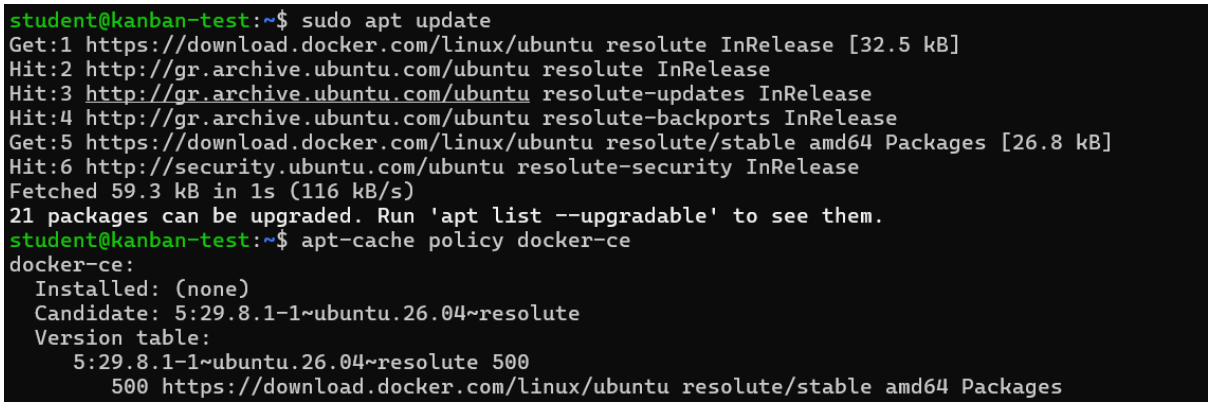
Добавим ключ GPG для официального репозитория Docker в систему:



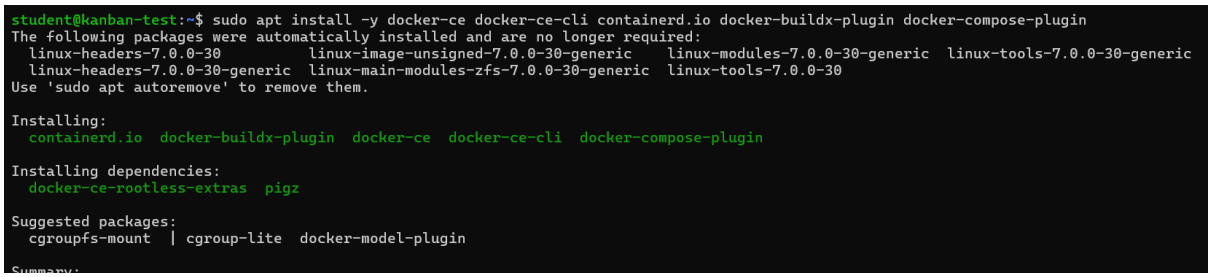
Добавим репозиторий Docker в источники APT:



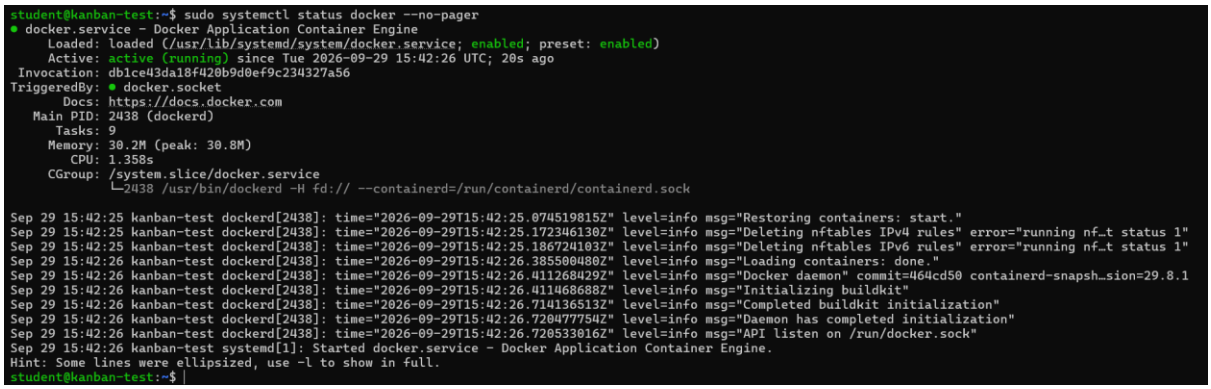
Обновим список пакетов и убедимся, что установка будет выполняться из репозитория Docker:



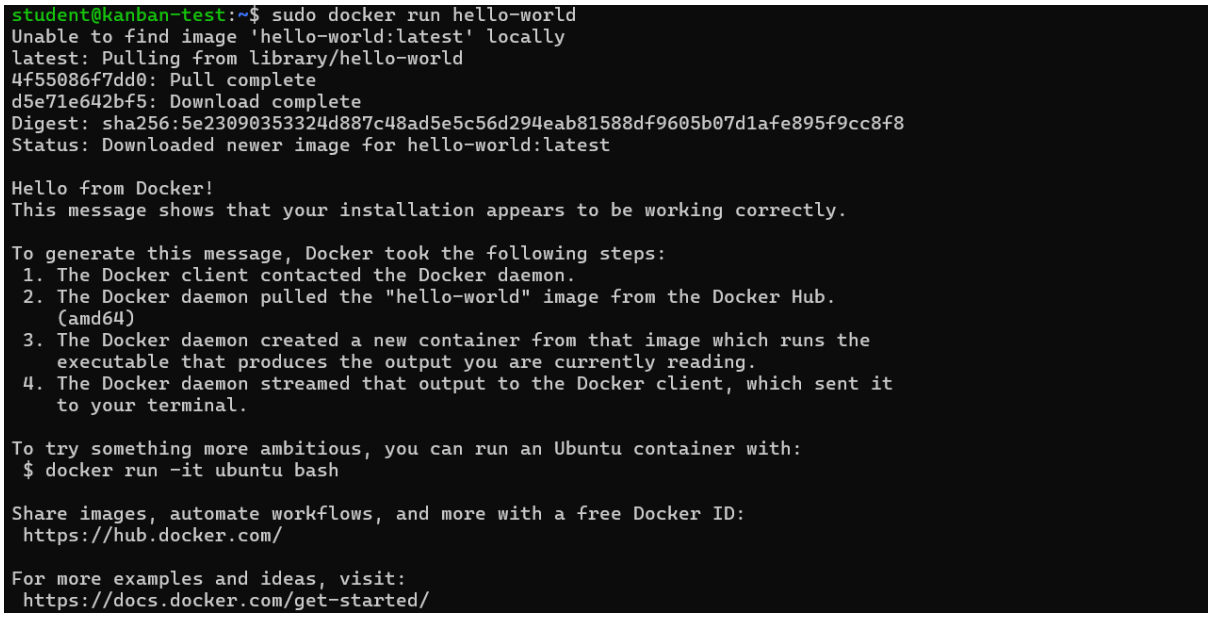
Установим Docker Engine и плагин Docker Compose:



Проверим, что служба Docker запущена:



Чтобы проверить доступ к Docker Hub и запуск контейнера, выполним команду sudo docker run hello-world:



2. На усмотрение студента выбрать Docker-образ, который будет являться основой для разрабатываемого приложения.

Для прототипа на Python и FastAPI выберем официальный Docker-образ python:3.14-slim. Образ содержит интерпретатор Python и минимальное окружение для установки зависимостей приложения.

3. Получить образ на TEST-виртуальную машину.

Получим выбранный образ на kanban-test с помощью docker pull и выведем список загруженных образов:



4. Добавить в корень проекта файл Dockerfile, внутри которого происходит сборка и запуск проекта из исходных файлов.

Перейдём в корень проекта и проверим наличие исходных файлов:

```
student@kanban-test:~$ cd ~/kanban-board
student@kanban-test:~/kanban-board$ pwd
/home/student/kanban-board
student@kanban-test:~/kanban-board$ ls
README.md  __pycache__  docs  main.py  requirements.txt
student@kanban-test:~/kanban-board$
```

Создадим Dockerfile. В нём зададим базовый образ, рабочий каталог, установку зависимостей и запуск сервера Uvicorn на порту 8000 внутри контейнера:

```
student@kanban-test:~/kanban-board$ cat > Dockerfile <<'EOF'
FROM python:3.14-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

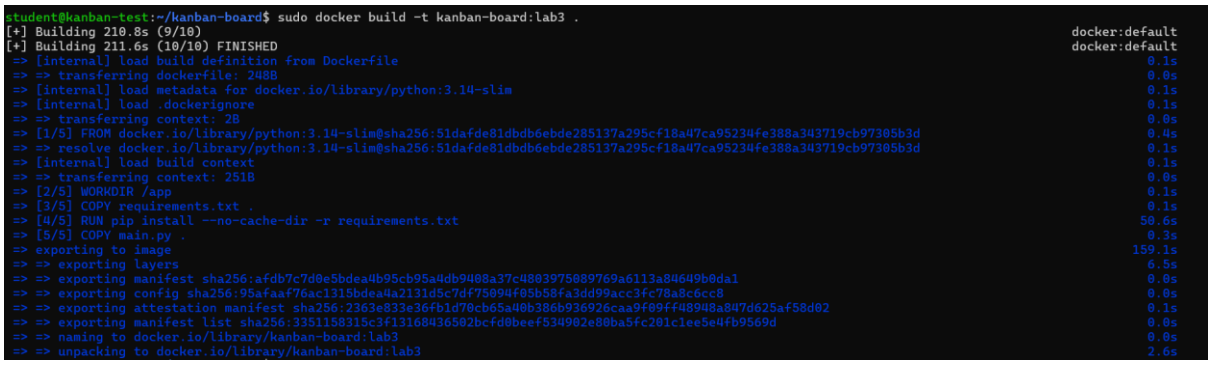
COPY main.py .

EXPOSE 8000

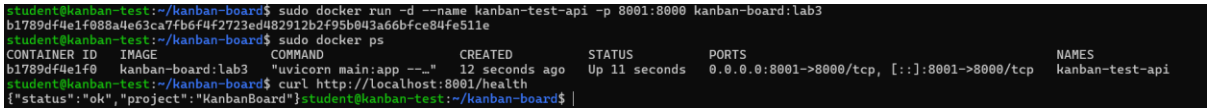
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
EOF
student@kanban-test:~/kanban-board$ S
```

5. Собрать приложение внутри Docker, запустить контейнер из полученного образа.

Соберём образ приложения командой sudo docker build -t kanban-board:lab3 .:



Запустим контейнер kanban-test-api с публикацией порта 8001 виртуальной машины на порт 8000 контейнера. Проверим состояние контейнера и ответ маршрута /health:

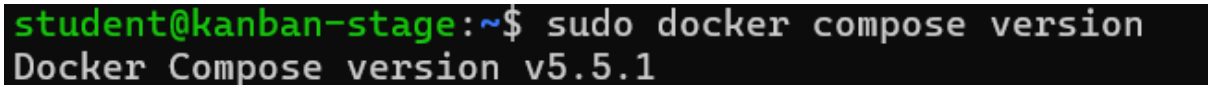


Контейнер запущен. Приложение вернуло статус ok и имя проекта KanbanBoard.

Часть 2

1. Установить на все ВМ утилиту docker-compose.

Плагин Docker Compose установлен вместе с Docker Engine. Для работы используем команду docker compose. На стенде kanban-stage выведем версию установленного плагина:



2. На выбор студента подобрать подходящую для приложения СУБД и получить с Docker Hub её официальный либо сторонний образ. В случае, если приложение не использует БД, выбрать любой сторонний внешний сервис, используемый приложением.

В соответствии со стеком проекта выберем PostgreSQL. Получим официальный образ postgres:17 на тестовый стенд:

```
student@kanban-test:~/kanban-board$ sudo docker pull postgres:17
17: Pulling from library/postgres
bed9376b51dc: Download complete
0725a8be915e: Pull complete
71f55f60a786: Pull complete
bed9376b51dc: Pull complete
752819419f50: Pull complete
8b51b9245051: Pull complete
73872941ba23: Download complete
498b9ff7d682: Pull complete
755a851127eb: Pull complete
73872941ba23: Pull complete
bbe7b3469b20: Extracting 16 s
bbe7b3469b20: Extracting 17 s
bbe7b3469b20: Pull complete
3feaebf93d15: Pull complete
2f9b6cf2e417: Pull complete
50e228e2fe6d: Pull complete


Digest: sha256:d74eeac9a635390a49bc21bd49fccd973de707e2a53a76ac49b552b8712ec46f
Status: Downloaded newer image for postgres:17
docker.io/library/postgres:17
```

Выведем загруженный образ PostgreSQL:

```
student@kanban-test:~/kanban-board$ sudo docker images postgres
IMAGE         ID             DISK USAGE   CONTENT SIZE   EXTRA
postgres:17   d74eeac9a635       646MB          167MB
student@kanban-test:~/kanban-board$
```

3. В корне проекта создать файл docker-compose.yaml, в котором описать параметры для сборки и запуска разрабатываемого приложения, а также запуска стороннего сервиса, выбранного студентом.

Перейдём в каталог ~/kanban-board. Добавим драйвер psycopg для подключения приложения к PostgreSQL:

```
student@kanban-test:~/kanban-board$ cat > requirements.txt <<'EOF'
fastapi[standard]
psycopg[binary]
EOF
```

В прототип добавим маршруты POST /cards и GET /cards для создания и чтения карточек. Параметры подключения передаются через переменную окружения DATABASE_URL:

Создадим файл .env с именем базы данных, пользователем и сгенерированным паролем. Файл .env исключим из Git:

```
student@kanban-test:~/kanban-board$ cat > .env <<EOF
POSTGRES_DB=kanban
POSTGRES_USER=kanban
POSTGRES_PASSWORD=$(openssl rand -hex 16)
EOF
```

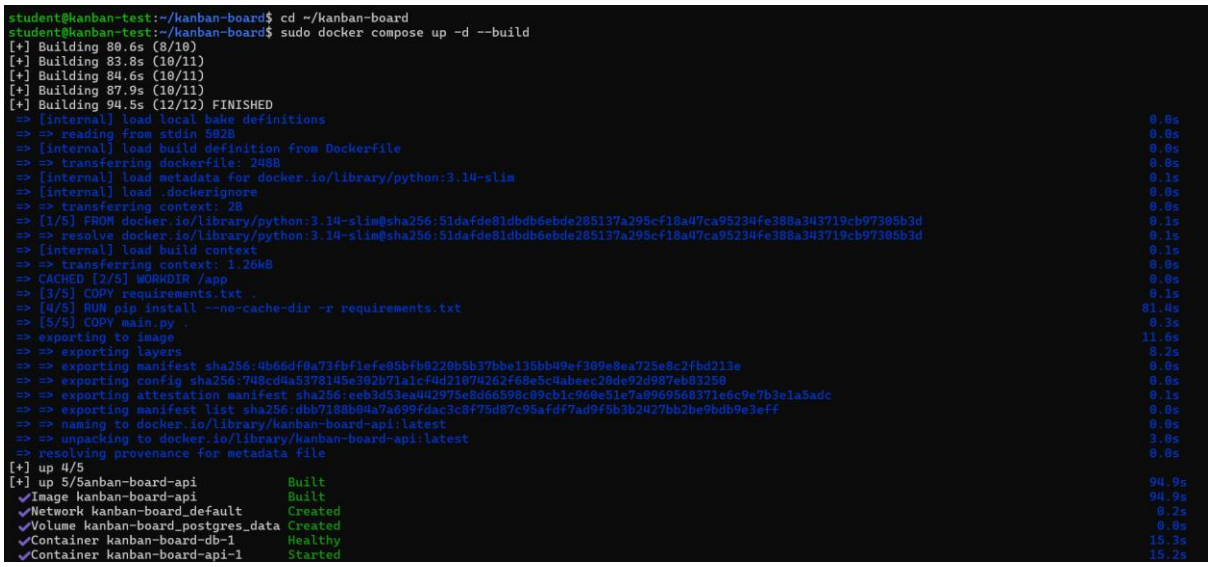
Создадим docker-compose.yaml с сервисами db и api. Для файлов PostgreSQL укажем постоянный том postgres_data. Приложение запускается после успешной проверки готовности БД и доступно на порту 8002 виртуальной машины:

```
student@kanban-test:~/kanban-board$ cat docker-compose.yaml
sudo docker compose config --quiet
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    build: .
    ports:
      - "8002:8000"
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
    depends_on:
      db:
        condition: service_healthy

volumes:
  postgres_data:
student@kanban-test:~/kanban-board$
```

Запустим Docker Compose со сборкой образа приложения:



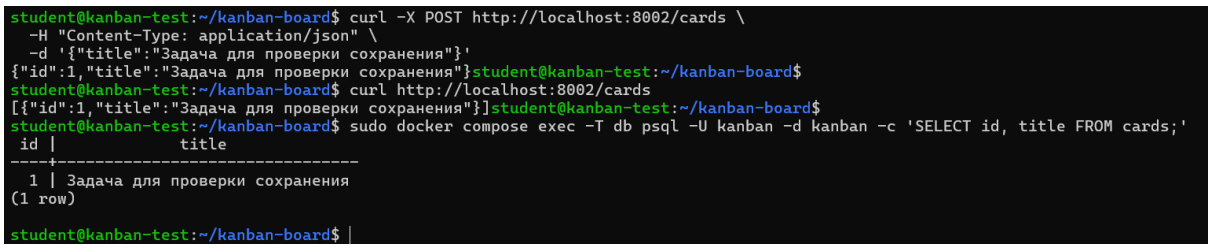
4. Запустить приложение и продемонстрировать его работу со сторонним сервисом.

Проверим запущенные сервисы и выполним запросы к /health и /cards. Сервис PostgreSQL находится в состоянии healthy. Пустой список карточек получен приложением из БД:



5. Внести изменения, требующие записи информации в файлы приложения — добавить записи в базу данных через приложение, создать файл и другие.

Через POST /cards добавим карточку «Задача для проверки сохранения». Получим список карточек через API и проверим запись непосредственно в таблице PostgreSQL:



6. Остановить и запустить приложение ещё раз, продемонстрировать, что изменения, внесённые в ходе работы, были сохранены.

Остановим приложение и удалим контейнеры командой docker compose down. Постоянный том PostgreSQL сохраняется. Затем повторно запустим сервисы командой docker compose up -d:

```
student@kanban-test:~/kanban-board$ sudo docker compose up -d
sudo docker compose ps
[+] up 3/3
 ✔Network kanban-board_default Created                                                                                0.1s
 ✔Container kanban-board-db-1  Healthy                                                                                6.6s
 ✔Container kanban-board-api-1 Started                                                                                6.7s
NAME                IMAGE               COMMAND                  SERVICE   CREATED         STATUS                   PORTS
kanban-board-api-1  kanban-board-api    "uvicorn main:app --…"   api       7 seconds ago   Up Less than a second    0.0.0.0:8002->8000/tcp, [::]:8002->8000/
tcp
kanban-board-db-1   postgres:17         "docker-entrypoint.s…"   db        7 seconds ago   Up 6 seconds (healthy)   5432/tcp
```

После перезапуска запросим список карточек:

```
student@kanban-test:~/kanban-board$ curl http://localhost:8002/cards
[{"id":1,"title":"Задача для проверки сохранения"}]student@kanban-test:~/kanban-board$
student@kanban-test:~/kanban-board$
```

В ответе сохранилась ранее созданная карточка с идентификатором 1. Сохранность данных после пересоздания контейнеров обеспечена томом postgres_data.

ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ

Официальная документация по проекту «KanbanBoard» - техническое задание и сопутствующие отчётные документы - предоставляется преподавателю в электронном виде в формате PDF. Исходные файлы, Dockerfile, конфигурация Docker Compose и техническая документация в формате Markdown размещаются в Git-репозитории проекта: https://github.com/icelanced/kanban-board. Файл .env с паролем базы данных не включается в репозиторий.

ВОПРОСЫ ДЛЯ ИЗУЧЕНИЯ

1. Каким образом Docker обеспечивает изоляцию контейнеров от хостовой ОС и друг от друга?

Docker использует механизмы ядра Linux: namespaces разделяют процессы, сетевые интерфейсы, точки монтирования и другие ресурсы, а cgroups позволяют ограничивать потребление процессора и памяти. Контейнеры имеют собственную файловую систему, но используют общее ядро хостовой ОС. Дополнительную защиту обеспечивают ограничения прав и системных вызовов.

2. Какие параметры требуется передать контейнеру Docker при запуске, чтобы его сетевой адаптер был тождественен сетевому адаптеру хостовой ОС, и почему?

При запуске на Linux нужно передать параметр --network host, например: sudo docker run --network host kanban-board:lab3. Контейнер использует сетевое пространство имён хоста: его интерфейсы, IP-адреса и порты. Отдельный IP контейнеру не назначается, а публикация портов через -p не требуется. В вашей работе хостом Docker является Ubuntu внутри ВМ

3. Какую последовательность команд требуется ввести, чтобы при помощи Docker Compose сначала собрать приложение, затем запустить его в фоновом режиме и затем подключиться к его выводу stdout и stderr?

В каталоге с docker-compose.yaml выполнить:

sudo docker compose build, sudo docker compose up -d, sudo docker compose logs -f

Первая команда собирает образы приложения, вторая запускает сервисы в фоновом режиме, третья показывает их stdout и stderr и отслеживает новые сообщения. Нажатие Ctrl+C завершает просмотр логов; контейнеры продолжают работать.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Методические указания к лабораторной работе № 3 «Контейнеризация» по дисциплине «Автоматизация процессов жизненного цикла программных средств». — Москва, 2024.

2. Техническое задание проекта «KanbanBoard». Лабораторная работа № 1. — Москва, 2026.

3. Docker Documentation. Установка Docker Engine на Ubuntu. — URL: https://docs.docker.com/engine/install/ubuntu/ (дата обращения: 30.09.2026).

4. Docker Documentation. Docker Compose. — URL: https://docs.docker.com/compose/ (дата обращения: 30.09.2026).

5. PostgreSQL — Docker Official Image. — URL: https://hub.docker.com/_/postgres (дата обращения: 30.09.2026).

6. Репозиторий проекта KanbanBoard. — URL: https://github.com/icelanced/kanban-board (дата обращения: 30.09.2026).